

Exercise 1

For the following schedule,

- Construct the precedence graph
- Is the schedule conflict-serializable?
- If so, what are all the equivalent serial schedules? Prove this by a series of local swaps.

$r_1(A); r_2(A); r_3(B); w_1(A); r_2(C); r_2(B); w_2(B); w_1(C)$

Solution

$r_1(A); r_2(A); r_3(B); w_1(A); r_2(C); r_2(B); w_2(B); w_1(C)$

$2 \preceq 1$ (twice) and $3 \prec 2$

Serial execution: T_3, T_2, T_1

Exercise 2

For the following schedule,

- Construct the precedence graph
- Is the schedule conflict-serializable?
- If so, what are all the equivalent serial schedules? Prove this by a series of local swaps.

$r_1(A); w_1(B); r_2(B) : w_2(C); r_3(C); w_3(A)$

Solution

$r_1(A); w_1(B); r_2(B) : w_2(C); r_3(C); w_3(A)$

$1 \preceq 3$, $1 \prec 2$ and $2 \prec 3$

Serial execution: T_1, T_2, T_3

Exercise 3

For the following schedule,

- Construct the precedence graph
- Is the schedule conflict-serializable?
- If so, what are all the equivalent serial schedules? Prove this by a series of local swaps.

$w_3(A); r_1(A); w_1(B); r_2(B); w_2(C); r_3(C);$

Solution

$w_3(A); r_1(A); w_1(B); r_2(B); w_2(C); r_3(C);$

$3 \preceq 1, 1 \prec 2$ and $2 \prec 3$

Not serializable

Exercise 4

For the following schedule,

- Construct the precedence graph
- Is the schedule conflict-serializable?
- If so, what are all the equivalent serial schedules? Prove this by a series of local swaps.

$r_1(A); r_2(A); w_1(B); w_2(B); r_1(B); r_2(B); w_2(C); w_1(D)$

Solution

$r_1(A); r_2(A); w_1(B); w_2(B); r_1(B); r_2(B); w_2(C); w_1(D)$

$1 \preceq 2, 2 \prec 1$ etc.

Not serializable

Exercise 5

For the following schedule,

- Construct the precedence graph
- Is the schedule conflict-serializable?
- If so, what are all the equivalent serial schedules? Prove this by a series of local swaps.

$r_1(A); r_2(A); r_1(B); r_2(B); r_3(A); r_4(B); w_1(A); w_2(B)$

Solution

$r_1(A); r_2(A); r_1(B); r_2(B); r_3(A); r_4(B); w_1(A); w_2(B)$

$1 \preceq 2, 2 \prec 1$ etc.

Not serializable

Exercise 6–10

For each schedule in exercises 1–5, do the following

- List each individual transaction, with lock and unlock actions added, to maximize concurrency
- Write the full schedule, with lock and unlock actions added
- Describe what the scheduler does, when each action arrives

Solution 6

T_1 : $l(A); r(A); w(A); l(C); u(A); w(C); u(C)$

T_2 : $l(A); r(A); l(C); r(C); l(B); u(A); u(C); r(B); w(B); u(B)$

T_3 : $l(B); r(B); u(B)$

Schedule:

$l_1(A); r_1(A); l_2(A); r_2(A); l_3(B); r_3(B); u_3(B); w_1(A);$

$l_2(C); r_2(C); l_2(B); u_2(A); u_2(C); r_2(B);$

$w_2(B); u_2(B); l_1(C); u_1(A); w_1(C); u_1(C)$

Transaction 1 gets lock on A . Transaction 2 requests lock and is delayed. Transaction 3 gets and releases lock on B . The next line (Transaction 2) waits. Transaction 1 gets lock on C and releases lock on A , Transaction 2 now runs. Finally, transaction 1 completes and releases the lock on C

Solution 7

$T_1: l(A); r(A); l(B); u(A); w(B); u(B)$

$T_2: l(B); r(B); l(C); u(B); w(C); u(C)$

$T_3: l(C); r(C); l(A); u(C); w(A); u(A)$

Schedule:

$l_1(A); r_1(A); l_1(B); u_1(A); w_1(B); u_1(B); l_2(B); r_2(B); l_2(C); u_2(B);$
 $w_2(C); u_2(C); l_3(A); r_3(C); l_3(C); u_3(A); w_3(A); u_3(A)$

Transaction 1 gets locks on A and B , releases them and concludes. Transaction 2 then gets locks on B and C , releases them, and concludes. Transaction 3 gets a locks on C , releases both locks, and concludes. Once 3 releases the lock on A , transaction 1 can run. Note that 1 has been delayed, resulting in a serializable sequence.

Solution 8

T_1 : $l(A); r(A); l(B); u(A); w(B); u(B)$

T_2 : $l(B); r(B); l(C); u(B); w(C); u(C)$

T_3 : $l(A); w(A); l(C); u(A); r(C); u(C)$

Schedule

$l_3(A); w_3(A); l_1(A); r_1(A); l_1(B); u_1(A); w_1(B); u_1(B); l_2(B); r_2(B);$
 $l_2(C); u_2(B); w_2(C); u_2(C); l_3(C); r_3(C); u_3(C)$

Transaction 3 gets a lock on A . Transaction 1 waits for 3 to release the lock. Transaction 2 gets locks on B and C and releases

Solution 9

T_1 : $l(A); r(A); l(B); w(B); r(B); l(D); u(B); u(A); w(D); u(D)$

T_2 : $l(A); r(A); l(B); w(B)r(B); l(C); u(A); u(B); w(C); u(C)$

Schedule

$l_1(A); r_1(A); l_2(A); r_2(A); l_1(B); w_1(B); l_2(B); w_2(B);$
 $r_1(B); r_2(B); l_2(C); u_2(A); u_2(B); w_2(C); u_2(C);$
 $l_1(D); u_1(A); u_1(B); w_1(D); u_1(D)$

Transaction 1 gets a lock on A and transaction 2 waits for one. Transaction 1 proceeds until it releases this lock (near the end) and then transaction 2 can proceed.

Solution 10

$T_1: l(A); r(A); l(B); r(B); l(B); w(A); u(A)$

$T_2: l(A); r(A); l(B); u(A); r(B); w(B); u(B)$

$T_3: l(A); r(A); u(A)$

$T_4: l(B); r(B); u(B)$

Schedule

$l_1(A); r_1(A); l_2(A); r_2(A); l_1(B); r_1(B); u_1(B); l_2(B); u_2(A); r_2(B);$
 $l_3(A); r_3(A); u_3(A); l_4(B); r_4(B); u_4(B); w_1(A); u_1(A); w_2(B); u_2(B)$

1 gets lock on A . 2 waits until 1 releases the lock. 1 gets lock on B and releases it. 3 also waits for a lock on A , while 4 gets lock on B and releases it. Now 1 terminates, and 3 and 2 (in either order) can get a lock on A and terminate